

project2

Sai Ram Vodnala

2025-03-27

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

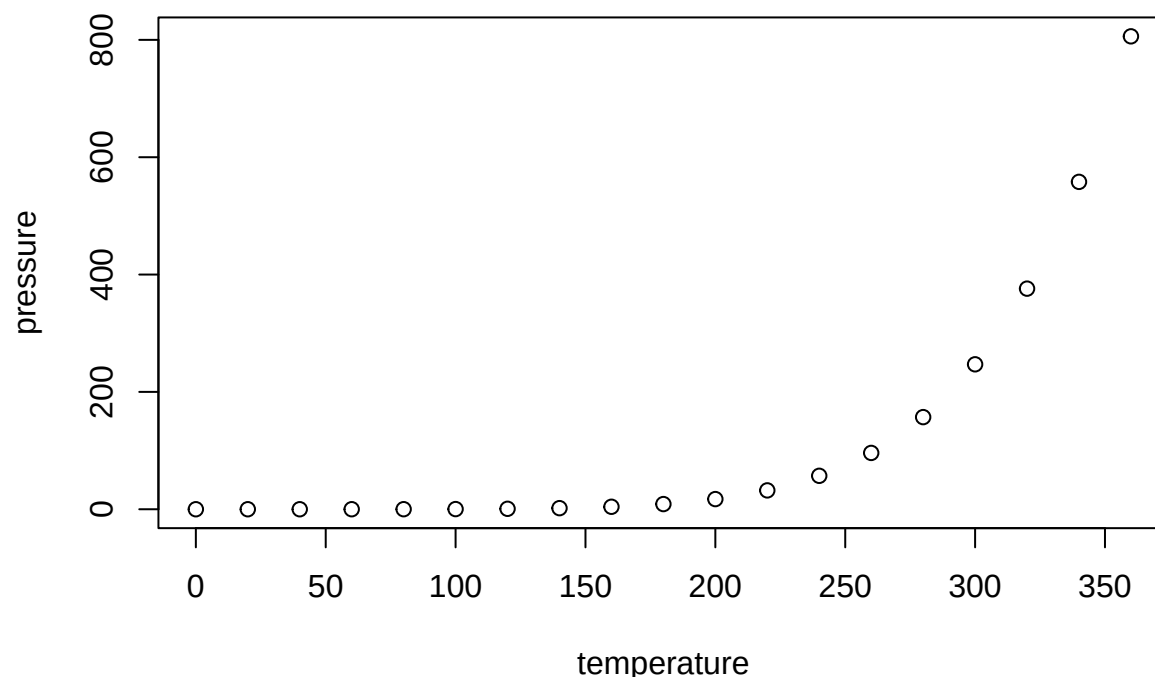
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean   : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.   :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```
# Loading the passcode dataset
```

```
Password_data <- read.csv("DSL-StrongPasswordData.csv", header = TRUE)
head>Password_data)
```

```
## subject sessionIndex rep H.period DD.period.t UD.period.t H.t DD.t.i
## 1 s002 1 1 0.1491 0.3979 0.2488 0.1069 0.1674
## 2 s002 1 2 0.1111 0.3451 0.2340 0.0694 0.1283
## 3 s002 1 3 0.1328 0.2072 0.0744 0.0731 0.1291
## 4 s002 1 4 0.1291 0.2515 0.1224 0.1059 0.2495
## 5 s002 1 5 0.1249 0.2317 0.1068 0.0895 0.1676
## 6 s002 1 6 0.1394 0.2343 0.0949 0.0813 0.1299
## UD.t.i H.i DD.i.e UD.i.e H.e DD.e.five UD.e.five H.five DD.five.Shift.r
## 1 0.0605 0.1169 0.2212 0.1043 0.1417 1.1885 1.0468 0.1146 1.6055
## 2 0.0589 0.0908 0.1357 0.0449 0.0829 1.1970 1.1141 0.0689 0.7822
## 3 0.0560 0.0821 0.1542 0.0721 0.0808 1.0408 0.9600 0.0892 0.6203
## 4 0.1436 0.1040 0.2038 0.0998 0.0900 1.0556 0.9656 0.0913 1.2564
## 5 0.0781 0.0903 0.1589 0.0686 0.0805 0.8629 0.7824 0.0742 0.8955
## 6 0.0486 0.0744 0.1412 0.0668 0.0863 0.9373 0.8510 0.0942 1.0896
## UD.five.Shift.r H.Shift.r DD.Shift.r.o UD.Shift.r.o H.o DD.o.a UD.o.a
## 1 1.4909 0.1067 0.7590 0.6523 0.1016 0.2136 0.1120
## 2 0.7133 0.1570 0.7877 0.6307 0.1066 0.1684 0.0618
## 3 0.5311 0.1454 0.7195 0.5741 0.1365 0.2931 0.1566
## 4 1.1651 0.1454 0.7550 0.6096 0.0956 0.1530 0.0574
```

```
## 5      0.8213      0.1243      0.7632      0.6389 0.0430 0.1975 0.1545
## 6      0.9954      0.1681      0.3716      0.2035 0.1154 0.1287 0.0133
##      H.a DD.a.n UD.a.n      H.n DD.n.l UD.n.l      H.l DD.l.Return UD.l.Return
## 1 0.1349 0.1484 0.0135 0.0932 0.3515 0.2583 0.1338      0.3509      0.2171
## 2 0.1412 0.2558 0.1146 0.1146 0.2642 0.1496 0.0839      0.2756      0.1917
## 3 0.1621 0.2332 0.0711 0.1172 0.2705 0.1533 0.1085      0.2847      0.1762
## 4 0.1457 0.1629 0.0172 0.0866 0.2341 0.1475 0.0845      0.3232      0.2387
## 5 0.1312 0.1582 0.0270 0.0884 0.2517 0.1633 0.0903      0.2517      0.1614
## 6 0.1272 0.1534 0.0262 0.0858 0.2528 0.1670 0.0792      0.2971      0.2179
##      H.Return
## 1      0.0742
## 2      0.0747
## 3      0.0945
## 4      0.0813
## 5      0.0818
## 6      0.0591
```

```
# Loading libraries
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.4.1
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      intersect, setdiff, setequal, union
```

```
library(readr)
```

```
## Warning: package 'readr' was built under R version 4.4.2
```

```
library(ggplot2)
```

```
library(caret) # Used for data partitioning
```

```
## Warning: package 'caret' was built under R version 4.4.3
```

```
## Loading required package: lattice
```

```
# To View structure
```

```
glimpse(Password_data)
```

```
## Rows: 20,400
```

```
## Columns: 34
```

```
## $ subject      <chr> "s002", "s002", "s002", "s002", "s002", "s002", "s002"~
```

```
## $ sessionIndex <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ rep <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
## $ H.period <dbl> 0.1491, 0.1111, 0.1328, 0.1291, 0.1249, 0.1394, 0.1064~
## $ DD.period.t <dbl> 0.3979, 0.3451, 0.2072, 0.2515, 0.2317, 0.2343, 0.2069~
## $ UD.period.t <dbl> 0.2488, 0.2340, 0.0744, 0.1224, 0.1068, 0.0949, 0.1005~
## $ H.t <dbl> 0.1069, 0.0694, 0.0731, 0.1059, 0.0895, 0.0813, 0.0866~
## $ DD.t.i <dbl> 0.1674, 0.1283, 0.1291, 0.2495, 0.1676, 0.1299, 0.1368~
## $ UD.t.i <dbl> 0.0605, 0.0589, 0.0560, 0.1436, 0.0781, 0.0486, 0.0502~
## $ H.i <dbl> 0.1169, 0.0908, 0.0821, 0.1040, 0.0903, 0.0744, 0.0800~
## $ DD.i.e <dbl> 0.2212, 0.1357, 0.1542, 0.2038, 0.1589, 0.1412, 0.1407~
## $ UD.i.e <dbl> 0.1043, 0.0449, 0.0721, 0.0998, 0.0686, 0.0668, 0.0607~
## $ H.e <dbl> 0.1417, 0.0829, 0.0808, 0.0900, 0.0805, 0.0863, 0.0789~
## $ DD.e.five <dbl> 1.1885, 1.1970, 1.0408, 1.0556, 0.8629, 0.9373, 0.7967~
## $ UD.e.five <dbl> 1.0468, 1.1141, 0.9600, 0.9656, 0.7824, 0.8510, 0.7178~
## $ H.five <dbl> 0.1146, 0.0689, 0.0892, 0.0913, 0.0742, 0.0942, 0.0855~
## $ DD.five.Shift.r <dbl> 1.6055, 0.7822, 0.6203, 1.2564, 0.8955, 1.0896, 1.2005~
## $ UD.five.Shift.r <dbl> 1.4909, 0.7133, 0.5311, 1.1651, 0.8213, 0.9954, 1.1150~
## $ H.Shift.r <dbl> 0.1067, 0.1570, 0.1454, 0.1454, 0.1243, 0.1681, 0.0948~
## $ DD.Shift.r.o <dbl> 0.7590, 0.7877, 0.7195, 0.7550, 0.7632, 0.3716, 0.3083~
## $ UD.Shift.r.o <dbl> 0.6523, 0.6307, 0.5741, 0.6096, 0.6389, 0.2035, 0.2135~
## $ H.o <dbl> 0.1016, 0.1066, 0.1365, 0.0956, 0.0430, 0.1154, 0.1233~
## $ DD.o.a <dbl> 0.2136, 0.1684, 0.2931, 0.1530, 0.1975, 0.1287, 0.1400~
## $ UD.o.a <dbl> 0.1120, 0.0618, 0.1566, 0.0574, 0.1545, 0.0133, 0.0167~
## $ H.a <dbl> 0.1349, 0.1412, 0.1621, 0.1457, 0.1312, 0.1272, 0.1318~
## $ DD.a.n <dbl> 0.1484, 0.2558, 0.2332, 0.1629, 0.1582, 0.1534, 0.1204~
## $ UD.a.n <dbl> 0.0135, 0.1146, 0.0711, 0.0172, 0.0270, 0.0262, -0.011~
## $ H.n <dbl> 0.0932, 0.1146, 0.1172, 0.0866, 0.0884, 0.0858, 0.0782~
## $ DD.n.l <dbl> 0.3515, 0.2642, 0.2705, 0.2341, 0.2517, 0.2528, 0.1999~
## $ UD.n.l <dbl> 0.2583, 0.1496, 0.1533, 0.1475, 0.1633, 0.1670, 0.1217~
## $ H.l <dbl> 0.1338, 0.0839, 0.1085, 0.0845, 0.0903, 0.0792, 0.0879~
## $ DD.l.Return <dbl> 0.3509, 0.2756, 0.2847, 0.3232, 0.2517, 0.2971, 0.2907~
## $ UD.l.Return <dbl> 0.2171, 0.1917, 0.1762, 0.2387, 0.1614, 0.2179, 0.2028~
## $ H.Return <dbl> 0.0742, 0.0747, 0.0945, 0.0813, 0.0818, 0.0591, 0.0890~
```

```
# How many unique subjects?
length(unique(Password_data$subject))
```

```
## [1] 51
```

```
# filtering column called 'rep' (typing repetition index)
filtered_data <- Password_data %>%
  filter(rep >= 31 & rep <= 50)

# to know shape of data
dim(filtered_data)
```

```
## [1] 8160 34
```

```
set.seed(123) # for reproducibility

# To Create stratified split
train_index <- createDataPartition(filtered_data$subject, p = 0.8, list = FALSE)
```

```
train_data <- filtered_data[train_index, ]
test_data <- filtered_data[-train_index, ]
```

For Viewing class distribution

```
table(train_data$subject)
```

```
##
## s002 s003 s004 s005 s007 s008 s010 s011 s012 s013 s015 s016 s017 s018 s019 s020
## 128 128 128 128 128 128 128 128 128 128 128 128 128 128 128 128
## s021 s022 s024 s025 s026 s027 s028 s029 s030 s031 s032 s033 s034 s035 s036 s037
## 128 128 128 128 128 128 128 128 128 128 128 128 128 128 128 128
## s038 s039 s040 s041 s042 s043 s044 s046 s047 s048 s049 s050 s051 s052 s053 s054
## 128 128 128 128 128 128 128 128 128 128 128 128 128 128 128 128
## s055 s056 s057
## 128 128 128
```

#Feature Engineering

#Creating new three columns total_h_time, total_dd_time, total_ud_time for both train and test sets

```
train_data <- train_data %>%
```

```
  mutate(
    total_h_time = rowSums(dplyr::select(., starts_with("H."))),
    total_dd_time = rowSums(dplyr::select(., starts_with("DD."))),
    total_ud_time = rowSums(dplyr::select(., starts_with("UD.")))
  )
```

```
test_data <- test_data %>%
```

```
  mutate(
    total_h_time = rowSums(dplyr::select(., starts_with("H."))),
    total_dd_time = rowSums(dplyr::select(., starts_with("DD."))),
    total_ud_time = rowSums(dplyr::select(., starts_with("UD.")))
  )
```

```
dim(train_data)
```

```
## [1] 6528 37
```

```
dim(test_data)
```

```
## [1] 1632 37
```

#Data Preprocessing

Standardizing all numeric columns (except subject) (Its Used for XGBoost and SVM)

```
library(caret)
```

```
preprocess <- preProcess(dplyr::select(train_data, -subject), method = c("center", "scale"))
```

```
train_scaled <- predict(preprocess, dplyr::select(train_data, -subject))
```

```
test_scaled <- predict(preprocess, dplyr::select(test_data, -subject))
```

```
# Adding subject to column back
train_scaled$subject <- train_data$subject
test_scaled$subject <- test_data$subject
```

```
#Baseline Model Building and Evaluation
```

```
#required libraries to be loaded
library(nnet)
library(MASS)
```

```
##
## Attaching package: 'MASS'

## The following object is masked from 'package:dplyr':
##
##      select
```

```
library(mclust)
```

```
## Warning: package 'mclust' was built under R version 4.4.2

## Package 'mclust' version 6.1.1
## Type 'citation("mclust")' for citing this R package in publications.
```

```
library(rpart)
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.4.3

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##      margin

## The following object is masked from 'package:dplyr':
##
##      combine
```

```
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.4.3
```

```
##
## Attaching package: 'xgboost'

## The following object is masked from 'package:dplyr':
##
## slice
```

```
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.4.3
```

```
library(tibble)
```

```
#Firstly selecting only three features and applying classification techniques
train_total <- train_data %>% dplyr::select(subject, total_h_time, total_dd_time, total_ud_time)
test_total <- test_data %>% dplyr::select(subject, total_h_time, total_dd_time, total_ud_time)

# To Make sure that 'subject' is a factor
train_total$subject <- as.factor(train_total$subject)
test_total$subject <- as.factor(test_total$subject)
```

```
#Model 1: Multinomial Logistic Regression
```

```
library(nnet)
```

```
log_model <- multinom(subject ~ ., data = train_total)
```

```
## # weights: 255 (200 variable)
## initial value 25666.957730
## iter 10 value 21873.440177
## iter 20 value 19687.619336
## iter 30 value 19079.710097
## iter 40 value 18079.201933
## iter 50 value 17486.731707
## iter 60 value 16911.095019
## iter 70 value 16293.874661
## iter 80 value 15697.674238
## iter 90 value 15228.506264
## iter 100 value 14800.899837
## final value 14800.899837
## stopped after 100 iterations
```

```
log_preds <- predict(log_model, newdata = test_total)
log_acc <- mean(log_preds == test_total$subject)
```

2. LDA

```
#Linear Discriminant Analysis
library(MASS)
```

```
lda_model <- lda(subject ~ ., data = train_total)
lda_preds <- predict(lda_model, newdata = test_total)$class
lda_acc <- mean(lda_preds == test_total$subject)
```

3. QDA

```
#Quadratic Discriminant Analysis
qda_model <- qda(subject ~ ., data = train_total)
qda_preds <- predict(qda_model, newdata = test_total)$class
qda_acc <- mean(qda_preds == test_total$subject)
```

4. MClustDA

```
#MclustDA discriminant analysis
library(mclust)

mclust_model <- MclustDA(train_total %>% dplyr::select(-subject), train_total$subject)
mclust_preds <- predict(mclust_model, test_total %>% dplyr::select(-subject))$classification
mclust_acc <- mean(mclust_preds == test_total$subject)
```

5. Decision Tree

```
library(rpart)

tree_model <- rpart(subject ~ ., data = train_total, method = "class")
tree_preds <- predict(tree_model, newdata = test_total, type = "class")
tree_acc <- mean(tree_preds == test_total$subject)
```

6. Random Forest

```
library(randomForest)

rf_model <- randomForest(subject ~ ., data = train_total)
rf_preds <- predict(rf_model, newdata = test_total)
rf_acc <- mean(rf_preds == test_total$subject)
```

7. XGBoost

```
library(xgboost)

# Preparing data f
x_train <- as.matrix(train_total %>% dplyr::select(total_ud_time, total_dd_time, total_h_time))
x_test <- as.matrix(test_total %>% dplyr::select(total_ud_time, total_dd_time, total_h_time))

y_train <- as.numeric(train_total$subject) - 1
y_test <- as.numeric(test_total$subject) - 1

xgb_model <- xgboost(data = x_train, label = y_train, objective = "multi:softmax",
                    num_class = length(unique(train_total$subject)),
                    nrounds = 30, verbose = 0)

xgb_preds <- predict(xgb_model, x_test)
xgb_acc <- mean(xgb_preds == y_test)
```

8.SVM


```
library(e1071)

svm_model <- svm(subject ~ ., data = train_total, kernel = "radial", cost = 1, gamma = 0.01)
svm_preds <- predict(svm_model, newdata = test_total)
svm_acc <- mean(svm_preds == test_total$subject)
```

Summary Table — Baseline Model Comparison

```
baseline_results <- tibble(
  Model = c("Multinomial Logistic", "LDA", "QDA", "MClustDA",
            "Decision Tree", "Random Forest", "XGBoost", "SVM"),
  Features = "3 total_features",
  Accuracy = c(log_acc, lda_acc, qda_acc, mclust_acc,
               tree_acc, rf_acc, xgb_acc, svm_acc)
)
#To print all baseline models
print(baseline_results)
```

```
## # A tibble: 8 x 3
##   Model           Features           Accuracy
##   <chr>           <chr>           <dbl>
## 1 Multinomial Logistic 3 total_features 0.331
## 2 LDA              3 total_features 0.325
## 3 QDA              3 total_features 0.359
## 4 MClustDA         3 total_features 0.384
## 5 Decision Tree    3 total_features 0.184
## 6 Random Forest    3 total_features 0.262
## 7 XGBoost          3 total_features 0.267
## 8 SVM              3 total_features 0.222
```

#Now Applying all features on eight models

```
# Performing PCA on training data (excluding subject)
pca_model <- prcomp(train_scaled %>% dplyr::select(-subject), center = TRUE, scale. = TRUE)

# Choosing top 17 PCs
n_pcs <- 17

train_pca <- as.data.frame(predict(pca_model, train_scaled %>% dplyr::select(-subject))[, 1:n_pcs])
train_pca$subject <- train_scaled$subject

test_pca <- as.data.frame(predict(pca_model, test_scaled %>% dplyr::select(-subject))[, 1:n_pcs])
test_pca$subject <- test_scaled$subject
```

1.Logistic Regression

```
log_model_pca <- multinom(subject ~ ., data = train_pca)
```

```
## # weights: 969 (900 variable)
## initial value 25666.957730
## iter 10 value 9482.405698
```

```
## iter 20 value 7562.015253
## iter 30 value 6971.721760
## iter 40 value 6570.907939
## iter 50 value 6243.786715
## iter 60 value 5679.926429
## iter 70 value 5296.481942
## iter 80 value 5018.419515
## iter 90 value 4875.796049
## iter 100 value 4739.355544
## final value 4739.355544
## stopped after 100 iterations
```

```
log_preds_pca <- predict(log_model_pca, newdata = test_pca)
log_acc_pca <- mean(log_preds_pca == test_pca$subject)
```

2.LDA

```
lda_model_pca <- lda(subject ~ ., data = train_pca)
lda_preds_pca <- predict(lda_model_pca, newdata = test_pca)$class
lda_acc_pca <- mean(lda_preds_pca == test_pca$subject)
```

3.QDA

```
qda_model_pca <- qda(subject ~ ., data = train_pca)
qda_preds_pca <- predict(qda_model_pca, newdata = test_pca)$class
qda_acc_pca <- mean(qda_preds_pca == test_pca$subject)
```

4.MclustDA

```
mclust_model_pca <- MclustDA(train_pca %>% dplyr::select(-subject), train_pca$subject)
mclust_preds_pca <- predict(mclust_model_pca, test_pca %>% dplyr::select(-subject))$classification
mclust_acc_pca <- mean(mclust_preds_pca == test_pca$subject)
```

Models with Full Raw Features 5. Decision Tree

```
tree_model_full <- rpart(subject ~ ., data = train_data, method = "class")
tree_preds_full <- predict(tree_model_full, newdata = test_data, type = "class")
tree_acc_full <- mean(tree_preds_full == test_data$subject)
```

6.Random Forest

```
train_data$subject <- as.factor(train_data$subject)
test_data$subject <- as.factor(test_data$subject)

rf_model_full <- randomForest(subject ~ ., data = train_data)
rf_preds_full <- predict(rf_model_full, newdata = test_data)
rf_acc_full <- mean(rf_preds_full == test_data$subject)
```

```
library(tibble)
```

```
advanced_results <- tibble(
```

```

Model = c(
  "Multinomial Logistic (PCA)",
  "LDA (PCA)",
  "QDA (PCA)",
  "MClustDA (PCA)",
  "Decision Tree (Full)",
  "Random Forest (Full)"
),
Features = c(rep("Top 17 PCs", 4), rep("Raw Features", 2)),
Accuracy = c(
  log_acc_pca,
  lda_acc_pca,
  qda_acc_pca,
  mclust_acc_pca,
  tree_acc_full,
  rf_acc_full
)
)

print(advanced_results)

```

```

## # A tibble: 6 x 3
##   Model           Features Accuracy
##   <chr>          <chr>      <dbl>
## 1 Multinomial Logistic (PCA) Top 17 PCs    0.803
## 2 LDA (PCA)         Top 17 PCs    0.722
## 3 QDA (PCA)         Top 17 PCs    0.763
## 4 MClustDA (PCA)    Top 17 PCs    0.790
## 5 Decision Tree (Full) Raw Features   0.494
## 6 Random Forest (Full) Raw Features   0.938

```

7.XGBoost

```

train_scaled$subject <- as.factor(train_scaled$subject)
test_scaled$subject  <- as.factor(test_scaled$subject)

# To Create a lookup table
subject_levels <- levels(train_scaled$subject)

# here Converting subjects to 0-based numeric labels
y_train_full <- as.integer(train_scaled$subject) - 1
y_test_full  <- as.integer(factor(test_scaled$subject, levels = subject_levels)) - 1

any(is.na(y_train_full)) # checking for nulls

```

```
## [1] FALSE
```

```
any(is.na(y_test_full)) # so Should be FALSE
```

```
## [1] FALSE
```

```
library(xgboost)

x_train_full <- as.matrix(train_scaled %>% dplyr::select(-subject))
x_test_full <- as.matrix(test_scaled %>% dplyr::select(-subject))

xgb_model_full <- xgboost(data = x_train_full, label = y_train_full, objective = "multi:softmax",
                          num_class = length(subject_levels),
                          nrounds = 30, verbose = 0)

xgb_preds_full <- predict(xgb_model_full, x_test_full)
xgb_acc_full <- mean(xgb_preds_full == y_test_full)

cat("XGBoost Accuracy (Full):", xgb_acc_full, "\n")
```

```
## XGBoost Accuracy (Full): 0.9099265
```

8.SVM

```
svm_model_full <- svm(subject ~ ., data = train_scaled, kernel = "radial", cost = 1, gamma = 0.01)
svm_preds_full <- predict(svm_model_full, newdata = test_scaled)
svm_acc_full <- mean(svm_preds_full == test_scaled$subject)
```

#Summary Table — Advanced Model Comparison

```
advanced_results <- tibble(
  Model = c("Multinomial Logistic (PCA)", "LDA (PCA)", "QDA (PCA)", "MClustDA (PCA)",
            "Decision Tree (Full)", "Random Forest (Full)", "XGBoost (Full)", "SVM (Full)"),
  Features = c(rep("Top 17 PCs", 4), rep("ALL Features", 4)),
  Accuracy = c(log_acc_pca, lda_acc_pca, qda_acc_pca, mclust_acc_pca,
               tree_acc_full, rf_acc_full, xgb_acc_full, svm_acc_full)
)

print(advanced_results)
```

```
## # A tibble: 8 x 3
##   Model           Features      Accuracy
##   <chr>          <chr>         <dbl>
## 1 Multinomial Logistic (PCA) Top 17 PCs      0.803
## 2 LDA (PCA)      Top 17 PCs      0.722
## 3 QDA (PCA)      Top 17 PCs      0.763
## 4 MClustDA (PCA) Top 17 PCs      0.790
## 5 Decision Tree (Full)    ALL Features    0.494
## 6 Random Forest (Full)    ALL Features    0.938
## 7 XGBoost (Full)         ALL Features    0.910
## 8 SVM (Full)            ALL Features    0.875
```

#Model Tuning

1.Random Forest

```

# To Control for 5-fold Cross validation
ctrl <- trainControl(method = "cv", number = 5)

# Tuning mtry (number of variables considered at each split)
set.seed(42)
rf_tuned <- train(subject ~ ., data = train_data,
                  method = "rf",
                  trControl = ctrl,
                  tuneGrid = expand.grid(mtry = c(5, 10, 15, 20)))

# printing the Best mtry value
print(rf_tuned$bestTune)

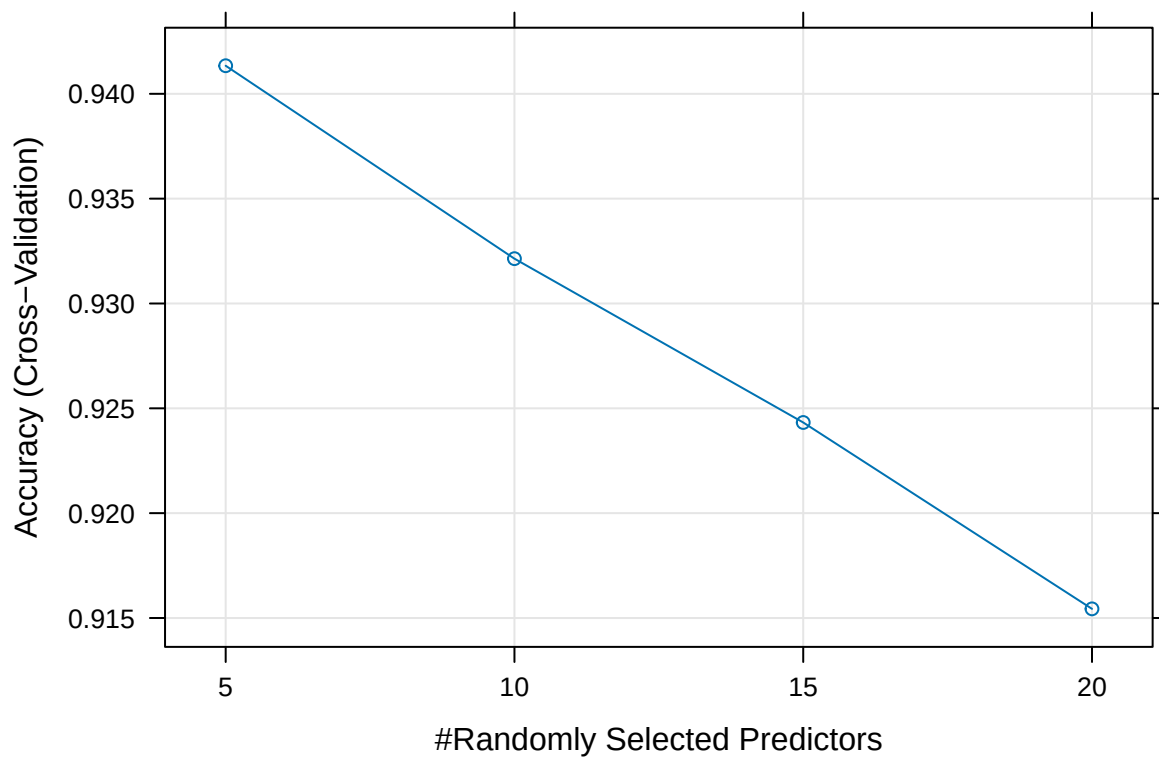
```

```

##      mtry
## 1      5

```

```
plot(rf_tuned)
```



```

# now Predicting on test set
rf_preds_tuned <- predict(rf_tuned, newdata = test_data)
rf_acc_tuned <- mean(rf_preds_tuned == test_data$subject)
cat("Random Forest Tuned Accuracy:", rf_acc_tuned, "\n")

```

```
## Random Forest Tuned Accuracy: 0.9381127
```

2.SVM

```
# Tried with less computational code
# Tuning Grid of C and sigma (gamma)
svm_grid <- expand.grid(
  C = c(1, 5),           # Here fewer cost values
  sigma = c(0.01, 0.1)   # fewer gamma values
)

set.seed(42)
svm_tuned <- train(subject ~ ., data = train_scaled,
  method = "svmRadial",
  preProcess = NULL,
  trControl = ctrl,
  tuneLength = 3)

svm_preds_tuned <- predict(svm_tuned, newdata = test_scaled)
svm_acc_tuned <- mean(svm_preds_tuned == test_scaled$subject)
cat("SVM Tuned Accuracy (Fast):", svm_acc_tuned, "\n")
```

SVM Tuned Accuracy (Fast): 0.8860294

3.XGBoost

```
# Encoding subject as factor first, then numeric labels
x_train <- as.matrix(train_scaled %>% dplyr::select(-subject))
x_test  <- as.matrix(test_scaled %>% dplyr::select(-subject))
y_train <- train_scaled$subject
y_test  <- test_scaled$subject

# caret compatible data
train_xgb <- train_scaled
train_xgb$subject <- as.factor(train_xgb$subject)

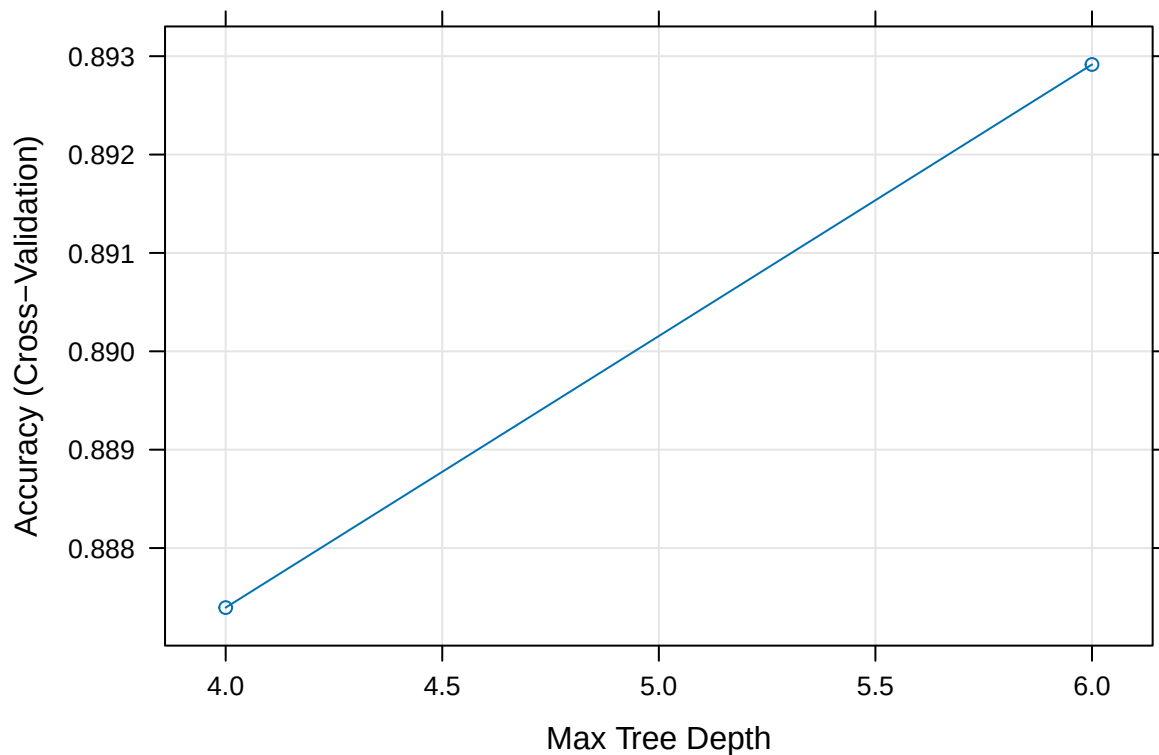
# Tuning the key parameters
xgb_grid <- expand.grid(
  nrounds = 30,
  max_depth = c(4, 6),
  eta = 0.1,
  gamma = 0,
  colsample_bytree = 1,
  min_child_weight = 1,
  subsample = 1
)

set.seed(42)
xgb_tuned <- train(subject ~ ., data = train_xgb,
  method = "xgbTree",
  trControl = ctrl,
  tuneGrid = xgb_grid,
  verbose = FALSE)

#printing the best parameters
print(xgb_tuned$bestTune)
```

```
## nrounds max_depth eta gamma colsample_bytree min_child_weight subsample
## 2      30      6 0.1      0      1      1      1
```

```
plot(xgb_tuned)
```



```
# now Predicting on test set
xgb_preds_tuned <- predict(xgb_tuned, newdata = test_scaled)
xgb_acc_tuned <- mean(xgb_preds_tuned == test_scaled$subject)
cat("XGBoost Tuned Accuracy (Fast):", xgb_acc_tuned, "\n")
```

```
## XGBoost Tuned Accuracy (Fast): 0.8897059
```

```
#Model Evaluation
```

```
library(caret)
library(dplyr)
library(ggplot2)
library(forcats)
```

```
## Warning: package 'forcats' was built under R version 4.4.2
```

```
#Considering the Random Forest is the best model
# Generating confusion matrix
```

```
conf_matrix <- confusionMatrix(rf_preds_tuned, test_data$subject)
```

```
# Printing full confusion matrix
```

```
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

## Prediction	s002	s003	s004	s005	s007	s008	s010	s011	s012	s013	s015	s016	s017
## s002	28	0	0	0	0	0	0	0	0	0	0	0	0
## s003	0	29	0	0	0	0	0	0	0	0	0	0	0
## s004	0	1	31	0	0	0	0	0	0	0	0	1	0
## s005	0	0	1	31	0	0	0	0	0	0	0	0	0
## s007	0	0	0	0	26	1	0	0	0	0	0	0	0
## s008	0	0	0	0	1	27	0	0	0	0	0	0	0
## s010	0	0	0	0	0	0	32	0	0	0	0	0	0
## s011	0	0	0	0	0	1	0	31	0	0	0	0	0
## s012	0	1	0	0	0	0	0	1	30	0	0	0	0
## s013	0	0	0	0	0	0	0	0	0	27	1	0	0
## s015	0	0	0	0	0	0	0	0	0	0	29	0	0
## s016	1	0	0	0	0	0	0	0	0	0	0	31	0
## s017	0	0	0	0	0	0	0	0	0	0	0	0	31
## s018	0	0	0	0	1	0	0	0	0	0	0	0	0
## s019	0	0	0	0	0	0	0	0	0	0	0	0	0
## s020	0	0	0	0	0	0	0	0	0	0	0	0	0
## s021	0	0	0	0	0	0	0	0	0	0	0	0	0
## s022	0	0	0	0	0	0	0	0	0	0	0	0	0
## s024	0	0	0	0	0	0	0	0	0	0	0	0	0
## s025	0	0	0	0	0	0	0	0	0	0	0	0	0
## s026	0	0	0	0	0	0	0	0	0	0	0	0	0
## s027	0	0	0	0	0	0	0	0	0	0	0	0	0
## s028	0	0	0	0	0	0	0	0	0	1	0	0	1
## s029	0	0	0	0	2	0	0	0	0	0	1	0	0
## s030	0	0	0	0	0	0	0	0	2	0	0	0	0
## s031	0	0	0	0	0	0	0	0	0	0	0	0	0
## s032	0	0	0	0	0	0	0	0	0	0	0	0	0
## s033	0	0	0	0	0	0	0	0	0	0	0	0	0
## s034	0	0	0	0	1	0	0	0	0	0	0	0	0
## s035	0	0	0	0	1	0	0	0	0	0	0	0	0
## s036	0	0	0	0	0	0	0	0	0	0	0	0	0
## s037	0	0	0	0	0	0	0	0	0	0	0	0	0
## s038	0	0	0	0	0	1	0	0	0	0	0	0	0
## s039	1	0	0	0	0	0	0	0	0	0	0	0	0
## s040	0	0	0	0	0	0	0	0	0	0	0	0	0
## s041	0	1	0	0	0	0	0	0	0	0	0	0	0
## s042	0	0	0	0	0	0	0	0	0	0	0	0	0
## s043	0	0	0	0	0	0	0	0	0	0	0	0	0
## s044	0	0	0	0	0	0	0	0	0	0	0	0	0
## s046	0	0	0	0	0	0	0	0	0	0	0	0	0
## s047	2	0	0	0	0	0	0	0	0	0	0	0	0
## s048	0	0	0	0	0	0	0	0	0	0	0	0	0
## s049	0	0	0	1	0	0	0	0	0	0	0	0	0
## s050	0	0	0	0	0	1	0	0	0	1	0	0	0

##	s051	0	0	0	0	0	0	0	0	0	0	0	0	
##	s052	0	0	0	0	0	0	0	0	0	0	0	0	
##	s053	0	0	0	0	0	0	0	0	0	1	0	0	
##	s054	0	0	0	0	0	1	0	0	0	0	0	0	
##	s055	0	0	0	0	0	0	0	0	0	0	0	0	
##	s056	0	0	0	0	0	0	0	0	0	0	0	0	
##	s057	0	0	0	0	0	0	0	0	3	0	0	0	
##	Reference													
##	Prediction	s018	s019	s020	s021	s022	s024	s025	s026	s027	s028	s029	s030	s031
##	s002	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s003	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s004	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s005	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s007	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s008	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s010	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s011	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s012	0	0	1	0	0	0	0	0	0	0	0	0	0
##	s013	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s015	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s016	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s017	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s018	31	0	0	0	0	0	0	0	0	0	0	0	0
##	s019	0	31	0	0	0	0	0	0	0	0	0	0	0
##	s020	0	0	28	0	0	0	0	0	0	0	0	0	0
##	s021	0	0	0	30	0	0	0	0	0	0	0	0	0
##	s022	0	0	0	0	32	0	0	0	0	0	0	0	0
##	s024	0	0	0	0	0	32	0	0	0	0	0	0	1
##	s025	0	0	0	0	0	0	32	0	0	0	0	0	0
##	s026	0	0	0	0	0	0	0	31	0	0	1	0	0
##	s027	0	0	0	0	0	0	0	0	32	0	0	0	0
##	s028	0	0	0	0	0	0	0	0	0	32	0	0	2
##	s029	0	0	0	0	0	0	0	0	0	0	31	0	0
##	s030	0	0	0	0	0	0	0	0	0	0	0	32	1
##	s031	0	1	0	0	0	0	0	0	0	0	0	0	25
##	s032	0	0	0	1	0	0	0	0	0	0	0	0	0
##	s033	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s034	0	0	1	0	0	0	0	0	0	0	0	0	0
##	s035	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s036	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s037	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s038	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s039	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s040	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s041	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s042	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s043	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s044	0	0	0	0	0	0	0	0	0	0	0	0	3
##	s046	0	0	0	0	0	0	0	1	0	0	0	0	0
##	s047	0	0	0	1	0	0	0	0	0	0	0	0	0
##	s048	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s049	1	0	0	0	0	0	0	0	0	0	0	0	0
##	s050	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s051	0	0	0	0	0	0	0	0	0	0	0	0	0

##	s052	0	0	0	0	0	0	0	0	0	0	0	0	
##	s053	0	0	0	0	0	0	0	0	0	0	0	0	
##	s054	0	0	2	0	0	0	0	0	0	0	0	0	
##	s055	0	0	0	0	0	0	0	0	0	0	0	0	
##	s056	0	0	0	0	0	0	0	0	0	0	0	0	
##	s057	0	0	0	0	0	0	0	0	0	0	0	0	
##	Reference													
##	Prediction	s032	s033	s034	s035	s036	s037	s038	s039	s040	s041	s042	s043	s044
##	s002	0	0	0	0	0	1	0	0	0	0	0	0	0
##	s003	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s004	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s005	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s007	1	0	0	0	0	0	0	0	0	0	0	0	0
##	s008	0	0	1	0	0	0	0	0	0	0	0	0	0
##	s010	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s011	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s012	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s013	1	0	0	0	0	0	0	0	0	0	0	0	0
##	s015	1	0	0	0	0	0	0	0	0	0	0	0	0
##	s016	0	0	0	0	0	0	0	0	1	0	0	0	0
##	s017	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s018	1	0	0	0	0	0	0	0	0	0	0	0	0
##	s019	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s020	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s021	2	0	0	0	0	0	0	0	0	0	0	0	0
##	s022	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s024	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s025	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s026	1	0	0	0	0	0	0	0	0	0	0	0	0
##	s027	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s028	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s029	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s030	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s031	0	0	0	0	0	3	0	0	1	2	0	0	0
##	s032	19	0	1	0	0	0	0	0	0	0	0	0	0
##	s033	0	32	0	0	0	0	0	0	0	0	0	0	0
##	s034	0	0	30	0	0	0	0	0	0	0	0	0	0
##	s035	0	0	0	32	0	0	0	0	0	0	0	0	0
##	s036	0	0	0	0	32	0	0	0	0	0	0	0	0
##	s037	3	0	0	0	0	25	0	0	0	0	0	0	0
##	s038	0	0	0	0	0	0	31	0	0	0	0	0	0
##	s039	0	0	0	0	0	0	0	32	0	0	0	0	0
##	s040	1	0	0	0	0	0	0	0	30	0	0	0	0
##	s041	0	0	0	0	0	0	0	0	0	30	0	0	0
##	s042	0	0	0	0	0	0	0	0	0	0	32	0	0
##	s043	0	0	0	0	0	0	0	0	0	0	0	32	0
##	s044	0	0	0	0	0	0	1	0	0	0	0	0	32
##	s046	0	0	0	0	0	1	0	0	0	0	0	0	0
##	s047	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s048	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s049	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s050	2	0	0	0	0	1	0	0	0	0	0	0	0
##	s051	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s052	0	0	0	0	0	0	0	0	0	0	0	0	0

##	s053	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s054	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s055	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s056	0	0	0	0	0	0	0	0	0	0	0	0	0
##	s057	0	0	0	0	0	1	0	0	0	0	0	0	0
##	Reference													
##	Prediction	s046	s047	s048	s049	s050	s051	s052	s053	s054	s055	s056	s057	
##	s002	0	0	0	0	0	0	0	0	0	0	0	0	
##	s003	0	0	0	0	0	0	0	0	0	0	0	0	
##	s004	0	0	0	0	0	0	0	0	0	0	0	0	
##	s005	0	0	0	0	0	0	0	0	0	0	0	0	
##	s007	0	0	0	0	0	4	0	0	0	0	0	0	
##	s008	0	0	1	0	0	0	0	0	0	0	0	0	
##	s010	0	0	0	0	0	0	0	0	0	0	1	1	
##	s011	0	0	0	0	0	0	0	0	0	0	0	0	
##	s012	0	0	0	0	0	0	0	0	0	0	0	0	
##	s013	0	0	0	0	0	0	0	0	0	0	0	0	
##	s015	0	0	0	0	0	0	0	0	0	0	0	1	
##	s016	1	0	0	0	0	0	0	0	0	0	0	0	
##	s017	0	0	0	0	0	0	0	0	0	0	0	0	
##	s018	0	0	0	0	0	0	0	0	1	0	0	0	
##	s019	0	0	0	0	0	0	0	0	0	0	0	0	
##	s020	0	0	0	0	0	0	0	0	1	0	0	0	
##	s021	0	0	0	0	0	0	0	0	0	0	0	1	
##	s022	0	0	0	0	0	0	0	0	0	0	0	0	
##	s024	0	0	0	0	0	0	0	0	0	0	0	0	
##	s025	0	2	0	0	0	0	0	0	0	0	0	0	
##	s026	0	0	0	0	0	0	0	0	0	0	0	0	
##	s027	0	0	0	0	0	0	0	0	0	0	0	0	
##	s028	0	0	0	0	0	0	0	0	0	0	0	0	
##	s029	0	0	0	0	0	0	0	0	0	0	0	0	
##	s030	1	0	0	0	0	0	0	0	0	0	0	0	
##	s031	0	0	0	0	0	0	0	0	0	0	0	0	
##	s032	0	0	0	0	0	0	0	0	0	0	1	0	
##	s033	0	0	0	0	0	0	0	0	0	0	0	0	
##	s034	0	0	0	0	0	0	0	0	0	0	0	0	
##	s035	0	0	0	0	0	0	0	0	0	0	0	0	
##	s036	0	0	0	0	0	0	0	0	0	0	0	0	
##	s037	0	0	0	0	0	0	0	0	0	0	0	0	
##	s038	0	0	0	0	0	0	0	0	0	0	0	0	
##	s039	0	0	0	0	1	0	0	0	0	0	0	0	
##	s040	0	0	0	0	0	0	0	0	0	0	0	0	
##	s041	0	0	0	0	0	0	0	0	0	0	0	0	
##	s042	0	0	0	0	0	0	0	0	0	0	0	0	
##	s043	0	0	0	0	0	0	0	0	0	0	0	0	
##	s044	0	0	0	0	0	0	0	0	0	0	0	0	
##	s046	29	0	0	0	0	0	0	0	0	0	0	0	
##	s047	0	29	1	0	0	0	0	0	0	0	0	0	
##	s048	0	0	30	0	0	0	0	0	0	0	0	2	
##	s049	1	0	0	32	0	0	0	0	0	0	0	0	
##	s050	0	1	0	0	31	0	0	0	1	0	0	0	
##	s051	0	0	0	0	0	28	0	0	0	0	0	0	
##	s052	0	0	0	0	0	0	32	0	0	0	0	0	
##	s053	0	0	0	0	0	0	0	32	0	0	0	0	

```

##          s054      0      0      0      0      0      0      0      0      0      29      0      0      0
##          s055      0      0      0      0      0      0      0      0      0      0      31      0      0
##          s056      0      0      0      0      0      0      0      0      0      0      1      30      0
##          s057      0      0      0      0      0      0      0      0      0      0      0      0      27
##
## Overall Statistics
##
##              Accuracy : 0.9381
##              95% CI : (0.9253, 0.9493)
##      No Information Rate : 0.0196
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9369
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: s002 Class: s003 Class: s004 Class: s005
## Sensitivity          0.87500      0.90625      0.96875      0.96875
## Specificity          0.99938      1.00000      0.99875      0.99938
## Pos Pred Value       0.96552      1.00000      0.93939      0.96875
## Neg Pred Value       0.99750      0.99813      0.99937      0.99938
## Prevalence           0.01961      0.01961      0.01961      0.01961
## Detection Rate       0.01716      0.01777      0.01900      0.01900
## Detection Prevalence 0.01777      0.01777      0.02022      0.01961
## Balanced Accuracy    0.93719      0.95312      0.98375      0.98406
##
##              Class: s007 Class: s008 Class: s010 Class: s011
## Sensitivity          0.81250      0.84375      1.00000      0.96875
## Specificity          0.99625      0.99813      0.99875      0.99938
## Pos Pred Value       0.81250      0.90000      0.94118      0.96875
## Neg Pred Value       0.99625      0.99688      1.00000      0.99938
## Prevalence           0.01961      0.01961      0.01961      0.01961
## Detection Rate       0.01593      0.01654      0.01961      0.01900
## Detection Prevalence 0.01961      0.01838      0.02083      0.01961
## Balanced Accuracy    0.90437      0.92094      0.99938      0.98406
##
##              Class: s012 Class: s013 Class: s015 Class: s016
## Sensitivity          0.93750      0.84375      0.90625      0.96875
## Specificity          0.99813      0.99875      0.99875      0.99813
## Pos Pred Value       0.90909      0.93103      0.93548      0.91176
## Neg Pred Value       0.99875      0.99688      0.99813      0.99937
## Prevalence           0.01961      0.01961      0.01961      0.01961
## Detection Rate       0.01838      0.01654      0.01777      0.01900
## Detection Prevalence 0.02022      0.01777      0.01900      0.02083
## Balanced Accuracy    0.96781      0.92125      0.95250      0.98344
##
##              Class: s017 Class: s018 Class: s019 Class: s020
## Sensitivity          0.96875      0.96875      0.96875      0.87500
## Specificity          1.00000      0.99813      1.00000      0.99938
## Pos Pred Value       1.00000      0.91176      1.00000      0.96552
## Neg Pred Value       0.99938      0.99937      0.99938      0.99750
## Prevalence           0.01961      0.01961      0.01961      0.01961
## Detection Rate       0.01900      0.01900      0.01900      0.01716
## Detection Prevalence 0.01900      0.02083      0.01900      0.01777
## Balanced Accuracy    0.98438      0.98344      0.98438      0.93719

```

##	Class: s021	Class: s022	Class: s024	Class: s025
## Sensitivity	0.93750	1.00000	1.00000	1.00000
## Specificity	0.99813	1.00000	0.99938	0.99875
## Pos Pred Value	0.90909	1.00000	0.96970	0.94118
## Neg Pred Value	0.99875	1.00000	1.00000	1.00000
## Prevalence	0.01961	0.01961	0.01961	0.01961
## Detection Rate	0.01838	0.01961	0.01961	0.01961
## Detection Prevalence	0.02022	0.01961	0.02022	0.02083
## Balanced Accuracy	0.96781	1.00000	0.99969	0.99938
##	Class: s026	Class: s027	Class: s028	Class: s029
## Sensitivity	0.96875	1.00000	1.00000	0.96875
## Specificity	0.99875	1.00000	0.99750	0.99813
## Pos Pred Value	0.93939	1.00000	0.88889	0.91176
## Neg Pred Value	0.99937	1.00000	1.00000	0.99937
## Prevalence	0.01961	0.01961	0.01961	0.01961
## Detection Rate	0.01900	0.01961	0.01961	0.01900
## Detection Prevalence	0.02022	0.01961	0.02206	0.02083
## Balanced Accuracy	0.98375	1.00000	0.99875	0.98344
##	Class: s030	Class: s031	Class: s032	Class: s033
## Sensitivity	1.00000	0.78125	0.59375	1.00000
## Specificity	0.99750	0.99562	0.99813	1.00000
## Pos Pred Value	0.88889	0.78125	0.86364	1.00000
## Neg Pred Value	1.00000	0.99562	0.99193	1.00000
## Prevalence	0.01961	0.01961	0.01961	0.01961
## Detection Rate	0.01961	0.01532	0.01164	0.01961
## Detection Prevalence	0.02206	0.01961	0.01348	0.01961
## Balanced Accuracy	0.99875	0.88844	0.79594	1.00000
##	Class: s034	Class: s035	Class: s036	Class: s037
## Sensitivity	0.93750	1.00000	1.00000	0.78125
## Specificity	0.99875	0.99938	1.00000	0.99813
## Pos Pred Value	0.93750	0.96970	1.00000	0.89286
## Neg Pred Value	0.99875	1.00000	1.00000	0.99564
## Prevalence	0.01961	0.01961	0.01961	0.01961
## Detection Rate	0.01838	0.01961	0.01961	0.01532
## Detection Prevalence	0.01961	0.02022	0.01961	0.01716
## Balanced Accuracy	0.96813	0.99969	1.00000	0.88969
##	Class: s038	Class: s039	Class: s040	Class: s041
## Sensitivity	0.96875	1.00000	0.93750	0.93750
## Specificity	0.99938	0.99875	0.99938	0.99938
## Pos Pred Value	0.96875	0.94118	0.96774	0.96774
## Neg Pred Value	0.99938	1.00000	0.99875	0.99875
## Prevalence	0.01961	0.01961	0.01961	0.01961
## Detection Rate	0.01900	0.01961	0.01838	0.01838
## Detection Prevalence	0.01961	0.02083	0.01900	0.01900
## Balanced Accuracy	0.98406	0.99938	0.96844	0.96844
##	Class: s042	Class: s043	Class: s044	Class: s046
## Sensitivity	1.00000	1.00000	1.00000	0.90625
## Specificity	1.00000	1.00000	0.99750	0.99875
## Pos Pred Value	1.00000	1.00000	0.88889	0.93548
## Neg Pred Value	1.00000	1.00000	1.00000	0.99813
## Prevalence	0.01961	0.01961	0.01961	0.01961
## Detection Rate	0.01961	0.01961	0.01961	0.01777
## Detection Prevalence	0.01961	0.01961	0.02206	0.01900
## Balanced Accuracy	1.00000	1.00000	0.99875	0.95250

```
##          Class: s047 Class: s048 Class: s049 Class: s050
## Sensitivity      0.90625      0.93750      1.00000      0.96875
## Specificity      0.99750      0.99875      0.99813      0.99562
## Pos Pred Value   0.87879      0.93750      0.91429      0.81579
## Neg Pred Value   0.99812      0.99875      1.00000      0.99937
## Prevalence       0.01961      0.01961      0.01961      0.01961
## Detection Rate    0.01777      0.01838      0.01961      0.01900
## Detection Prevalence 0.02022      0.01961      0.02145      0.02328
## Balanced Accuracy 0.95188      0.96813      0.99906      0.98219
##          Class: s051 Class: s052 Class: s053 Class: s054
## Sensitivity      0.87500      1.00000      1.00000      0.90625
## Specificity      1.00000      1.00000      0.99938      0.99813
## Pos Pred Value   1.00000      1.00000      0.96970      0.90625
## Neg Pred Value   0.99751      1.00000      1.00000      0.99813
## Prevalence       0.01961      0.01961      0.01961      0.01961
## Detection Rate    0.01716      0.01961      0.01961      0.01777
## Detection Prevalence 0.01716      0.01961      0.02022      0.01961
## Balanced Accuracy 0.93750      1.00000      0.99969      0.95219
##          Class: s055 Class: s056 Class: s057
## Sensitivity      0.96875      0.93750      0.84375
## Specificity      1.00000      0.99938      0.99750
## Pos Pred Value   1.00000      0.96774      0.87097
## Neg Pred Value   0.99938      0.99875      0.99688
## Prevalence       0.01961      0.01961      0.01961
## Detection Rate    0.01900      0.01838      0.01654
## Detection Prevalence 0.01900      0.01900      0.01900
## Balanced Accuracy 0.98438      0.96844      0.92063
```

```
# Extracting class-wise stats
class_stats <- as.data.frame(conf_matrix$byClass)

# Add subject label
class_stats$subject <- rownames(class_stats)

# Keep main metrics
summary_stats <- class_stats %>%
  dplyr::select(subject, Precision = Precision, Recall = Recall, F1 = F1)

# View top/bottom performance classes
summary_stats %>% arrange(F1) %>% head()
```

```
##          subject Precision Recall      F1
## Class: s032 Class: s032 0.8636364 0.59375 0.7037037
## Class: s031 Class: s031 0.7812500 0.78125 0.7812500
## Class: s007 Class: s007 0.8125000 0.81250 0.8125000
## Class: s037 Class: s037 0.8928571 0.78125 0.8333333
## Class: s057 Class: s057 0.8709677 0.84375 0.8571429
## Class: s008 Class: s008 0.9000000 0.84375 0.8709677
```

```
summary_stats %>% arrange(desc(F1)) %>% head()
```

```
##          subject Precision Recall F1
## Class: s022 Class: s022      1      1  1
```

```
## Class: s027 Class: s027      1      1  1
## Class: s033 Class: s033      1      1  1
## Class: s036 Class: s036      1      1  1
## Class: s042 Class: s042      1      1  1
## Class: s043 Class: s043      1      1  1
```

```
#Now Creating a dataframe of actual vs. predicted
library(dplyr)
```

```
misclass_df <- data.frame(
  actual = test_data$subject,
  predicted = rf_preds_tuned
)
```

```
# Keeping only incorrect predictions
misclass_df <- misclass_df %>%
  filter(actual != predicted)
```

```
# To Count confusion pairs
confusion_pairs <- misclass_df %>%
  group_by(actual, predicted) %>%
  summarise(freq = n(), .groups = "drop") %>%
  arrange(desc(freq))
```

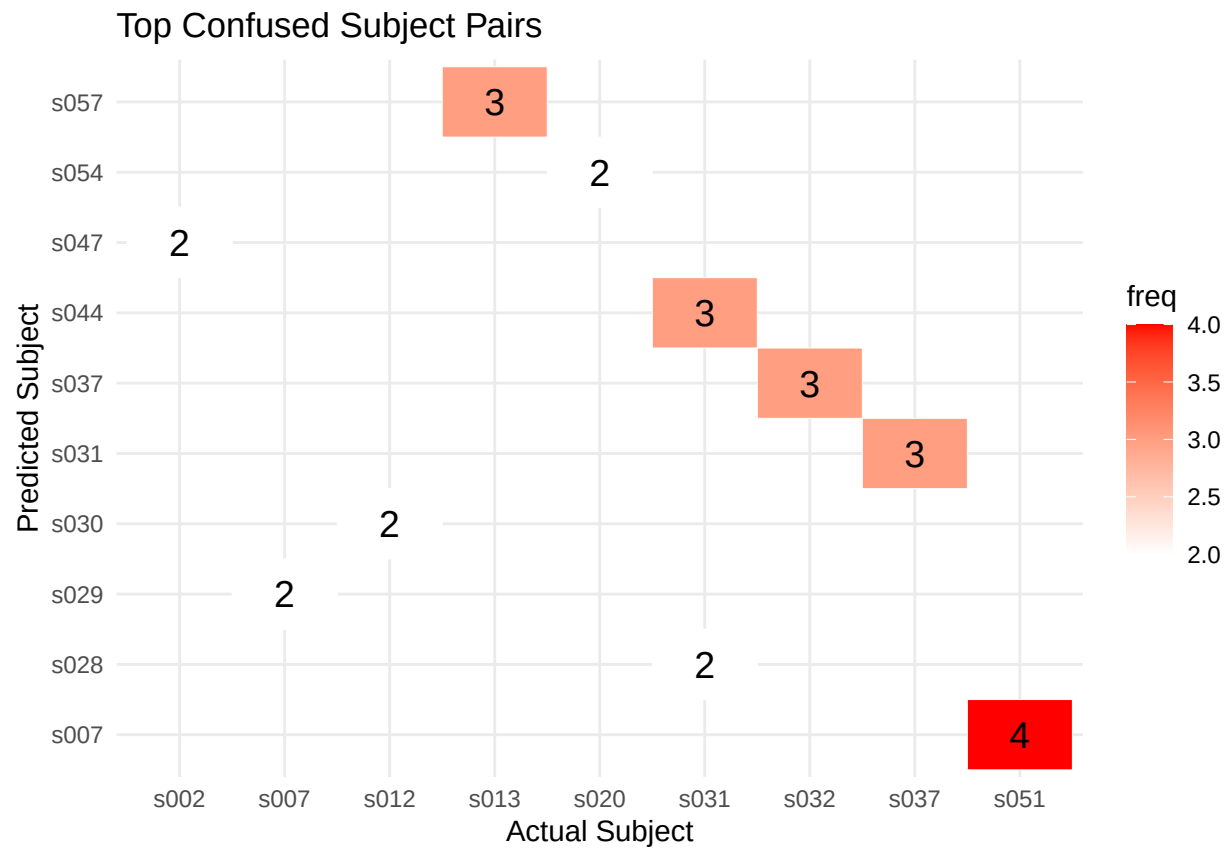
```
# To View top 10 most confused subject pairs
head(confusion_pairs, 10)
```

```
## # A tibble: 10 x 3
##   actual predicted freq
##   <fct> <fct>    <int>
## 1 s051  s007         4
## 2 s013  s057         3
## 3 s031  s044         3
## 4 s032  s037         3
## 5 s037  s031         3
## 6 s002  s047         2
## 7 s007  s029         2
## 8 s012  s030         2
## 9 s020  s054         2
## 10 s031  s028         2
```

```
library(ggplot2)
```

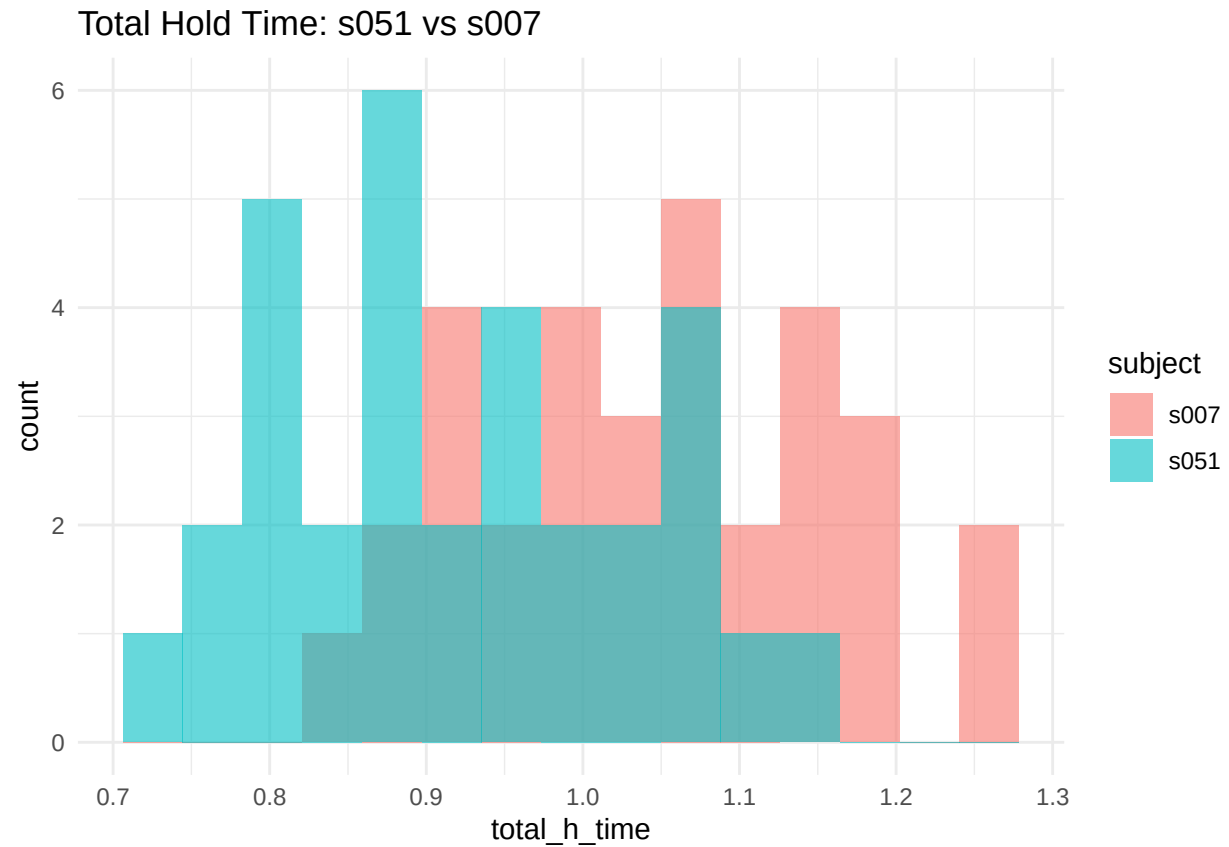
```
top_confusions <- head(confusion_pairs, 10)
```

```
ggplot(top_confusions, aes(x = actual, y = predicted, fill = freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = freq), size = 5) +
  scale_fill_gradient(low = "white", high = "red") +
  labs(title = "Top Confused Subject Pairs",
       x = "Actual Subject", y = "Predicted Subject") +
  theme_minimal()
```



```
confused_pair <- c("s051", "s007")

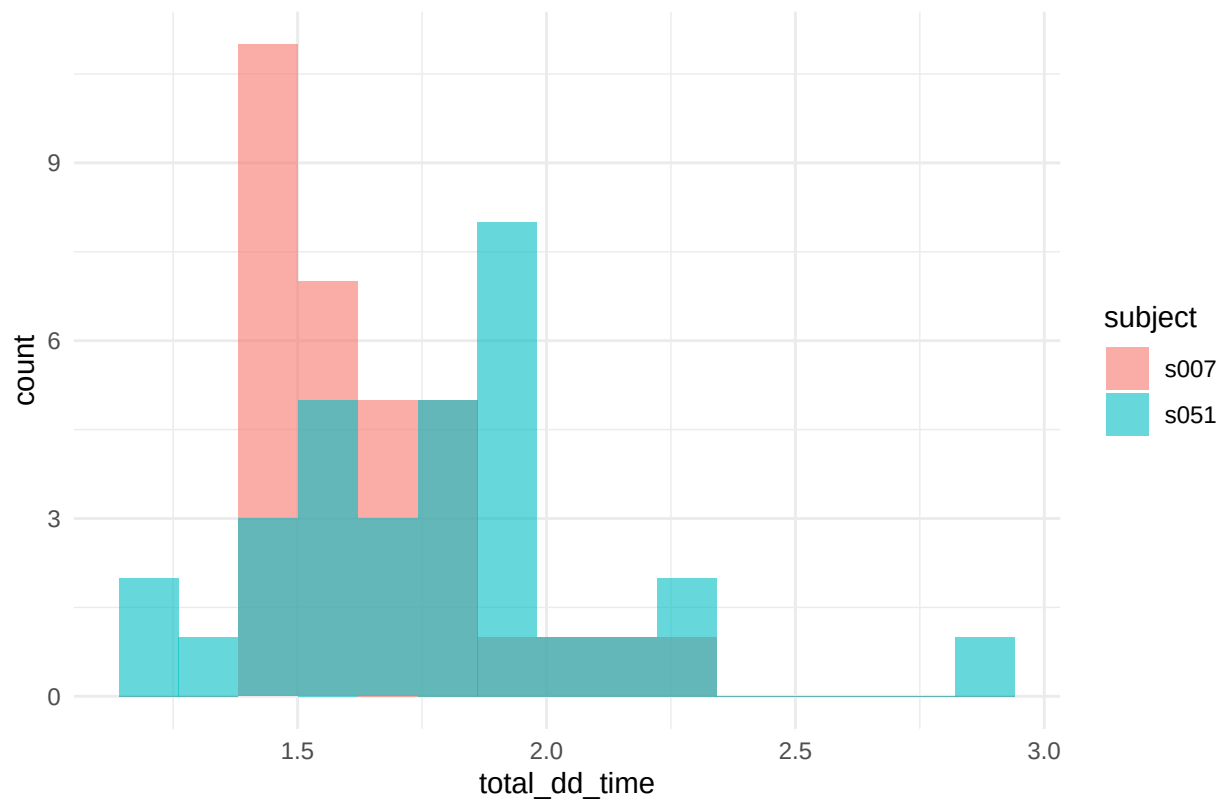
ggplot(test_data %>% filter(subject %in% confused_pair),
  aes(x = total_h_time, fill = subject)) +
  geom_histogram(alpha = 0.6, bins = 15, position = "identity") +
  labs(title = "Total Hold Time: s051 vs s007") +
  theme_minimal()
```

```
confused_pair <- c("s051", "s007")

ggplot(test_data %>% filter(subject %in% confused_pair),
  aes(x = total_dd_time, fill = subject)) +
  geom_histogram(alpha = 0.6, bins = 15, position = "identity") +
  labs(title = "Total DD Time: s051 vs s007") +
  theme_minimal()
```

Total DD Time: s051 vs s007



```
confused_pair <- c("s051", "s007")

ggplot(test_data %>% filter(subject %in% confused_pair),
       aes(x = total_ud_time, fill = subject)) +
  geom_histogram(alpha = 0.6, bins = 15, position = "identity") +
  labs(title = "Total UD Time: s051 vs s007") +
  theme_minimal()
```

Total UD Time: s051 vs s007

